

目录

目录	1
新增鸿蒙笔记	2
移植 或者内存经验	3
内存分析	4
例如，Linux系统/proc文件夹下的pid/maps接口	4
指令和avx	7
pragma GCC optimize("O3") gcc	8
ARM压栈的顺序依次为当前函数指针PC、返回指针LR、栈指针SP、栈基址FP、传入参数个数及指针、本地变量和临时变量	9
性能加速实践	10
未命名的页面(1)	12

新增鸿蒙笔记

Welcome to the graphic_graphic_3d wiki!

export OMP_NUM_THREADS=1 设置OMP线程为1 export BLAS_NUM_THREADS=1 设置BLAS库线程为1

export OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1 export OMPI_ALLOW_RUN_AS_ROOT=1 export OPAL_PREFIX=\$foo_path export LD_LIBRARY_PATH=\$foo_path

常用Linux命令行 lddisplay vgdisplay pvdisplay 系统和硬件相关 lscpu cat /proc/cpuinfo lsmem lspci iostat iotop dmidecode -t memory | grep Size

lscpu | grep Thread # 最简单的超线程判断 cat /proc/cpuinfo | grep "cores" | uniq #cpu逻辑核心数量 cat /proc/cpuinfo | grep "physical id" | sort | uniq #物理cpu个数 cat /proc/cpuinfo | grep "processor" | wc -l #cpu线程数量

df -lh du -lh --max-depth=1 lsblk zip -q -r -y 压缩的包名 被压的路径 unzip 解压的包名 解包 : tar xvf FileName.tar 打包 : tar cvf FileName.tar DirName

find / -type f -size +1000000k find . -xdev -type f -size +100M 查找大文件 find / -name "foo" 在/下查找名字为foo的文件

git pull (一句命令搞定) git clone -b 分支名 git config --global credential.helper store

git config --system user.name "foo" root git config --system user.email "f@f" root git config --system --list root git config --system http.sslVerify false root git config --system credential.helper store echo 10.1.204.218 codehub-g.huawei.com >> /etc/hosts echo 10.1.204.218 codehub-dg-g.huawei.com >> /etc/hosts

ldd addr2line -e 文件名 -f 0x5678 readelf c++filt 由C++符号查原始函数 objdump strip nm -A strings

移植 或者内存经验

移植时碰到的一点问题 在arm下需要关闭加成融合指令，pow函数精度调到最高 理论上来说这个指令能明显提升性能

进行仿真行业 高性能计算 需要的技术和手段： 1、mpi-分布式多进程通信（具体实现有intel mpi、open mpi） 2、矩阵库（具体实现有intel mkl、open blas、鲲鹏数学库） 3、vtk可视化库（仿真结果提取） 4、Cmake 5、线程技术，主要是高级线程库（具体实现有intel tbb、taskflow等） 6、Linux环境编程、Linux系统接口、Shell脚本 7、Python 8、稀疏矩阵求解器：Mumps 华为另有替代自研 9、性能分析，在代码内手动插桩、perf、intel vtune 10、Linux环境应用，Linux设备接口，逻辑卷，SWAP 11、计算机硬件基础，多CPU，大内存环境，NUMA 12、编译技术，一些安全编译选项的含义，可能影响计算精度的编译选项，后续可能从gcc切llvm 13、调试手段，跨平台调试，Linux gdb调试

[gdb GDB中文手册完美版.pdf](#)

这个文档会告诉你从磁盘读30个预览图，到显示器显示出来，花了多少时间

<https://static.googleusercontent.com/media/research.google.com/en/us/people/jeff/stanford-295-talk.pdf>

计算机体系结构 量化研究方法 <https://mxlol.gitbook.io/ji-suan-ji-ti-xi-jie-gou-liang-hua-yan-jiu-fang-fa-di-liu-ban-han-hua/di-yi-zhang-liang-hua-she-ji-he-fen-xi-de-ji-chu-zhi-shi/1.5-ji-cheng-dian-lu-zhong-gongl-he-neng-hao-de-fa-zhan-qu-shi/wei-chu-li-q-nei-de-neng-hao-he-gong-l>

vulkan <https://zhuanlan.zhihu.com/p/165341403>

强推intel vulkan utorial，看这个链接就行，不需要往下看我写的东西了

<https://www.intel.com/content/www/us/en/developer/articles/training/api-without-secrets-introduction-to-vulkan-part-1.html>

例如，Selecting Presentation Mode这一节，就很好的介绍了需要3缓冲的原因，以及送显模式如何平衡一些问题

而其他的文章（甚至是国内的文章），只会告诉你 用3缓冲就完事了 别问为什么

内存分析

最近处理了几个内存问题，回顾一下自己的开发生涯，实际上我碰到过许多奇怪的内存场景

但是我从来没有做过总结，以至于丢失了许多宝贵的调试经验

目前只有cpu堆栈内存的内容，gpu以后可能会补充

1、计算机和操作系统基础

(善用AI 这一段来自AI)

我先把内存的简单的分成3类：gpu、cpu、DMA（本质上还是物理内存条分出来的一块区域）

gpu内存即显存，其中的分区我暂时不感兴趣

DMA在图形学中主要是能让gpu绕过cpu直接读写主存的一块区域

cpu内存，这里特指cpu在处理进程的代码时，能够合法访问的进程空间

首先我们要对内存有深刻的了解，知道进程空间的概念，以及如何查看进程空间的范围

一个简单的问题，有进程A，有地址0x7fff29102，如果无法确定地址是否属于进程A，那还怎么分析下去呢？

一般情况下，操作系统会提供一些能够陷入内核态接口让我们得到进程的内存列表

例如，Linux系统/proc文件夹下的pid/maps接口

```
cat /proc/2617/maps cat /proc/${pidof com.example.foo}/maps
```

列表很长，其内容的构成，可以在需要时去谷歌了解

还有更高阶的原理，内核技术，cpu陷阱等，cpu是如何发现某个地址，进程是无权访问的？

这些技术可以在需要时去谷歌，感兴趣的朋友可以在周末有空时阅读

2、检查、刺探内存的手段

1. 调试器--debug工具 lldb gdb
2. 编译器--编译时增加编译标志 如-fstack-protector、-fsanitize=address等
3. 基于业务或代码的逻辑理解，手动加打印
4. Valgrind等三方库
5. 静态代码分析工具
6. 鸿蒙专属的hidumper和HiSmartPerf
7. 就不说了真这么厉害哪还用得着工具 直接改代码提交，解决问题就行，5)对动态场景基本没啥用
8. 应该是鸿蒙开发比较常用的工具

鸿蒙设备内置的hidumper命令的可以快速了解进程的内存，其原理就是使用系统接口获取内存数据

然后加一些处理，以便于人类(human)能够快速理解

(为什么要用human 因为我想扩展告诉你有些系统命令-h参数展开并不是help，有可能是human，即人类友好)

在鸿蒙设备的tty下(shell窗口)可以使用hidumper -h得到使用帮助

1)的比较像3)的高级版本，需要有对代码的逻辑有理解

一般情况是给业务代码打断点，通过断点查看变量的上下文仔细检查内存值，去分析问题

还有一个调试器功能是watch，指定最多2个内存地址，在地址被读写时提供断点

例如，监控栈变量，在栈变量被修改时检查上下文的内存，如果内存值不对，即找到了栈溢出的代码处

(同理也能对堆进行检查，缺点是不能全自动化，需要程序员手动操作)

```
// https://khorbushko.github.io/article/2021/05/27/the-lldb-debugger-part3-watchpoint.html
(lldb) p &inUse
(GLInt (*)[2]) $4 = 0x00007f8e7ce3e6b0

(lldb) watch set expression -- 0x00007f8e7ce3e6b0
(lldb) watch set variable inUse

// 可以这样进行读写监控
(lldb) watch set expression -w read_write -- 0x00007f8e7ce3e6b0
```

手段2)关于其他的内存监控的编译标志 以后用到其他的标志再补充

不同的编译标志，本质上都是在需要检查的地方，编译器插入cpu指令，去检查内存

但不同的标志，产生的指令逻辑不同，对程序的影响也就不同

-fstack-protector的原理，arm文档的描述是这样的：

在函数的栈顶插入一个值，如果栈被越界写了，这个值一定会变化，在函数退出时检查这个变量

```
// https://stackoverflow.com/questions/10325713/why-does-this-memory-address-fs0x28-fs0x28-have-a-random-value

// 函数开始时 使用rax保存一个随机值 作为保护变量
0x00007f8e7cf9a8d9 64 48 8b 04 25 28 00 00 00      movq    %fs:0x28, %rax
0x00007f953fd5a8e2 48 89 84 24 e0 00 00 00      movq    %rax, 0xe0(%rsp);

// 在函数结束时 把保护变量取到rcx并和rax比较，如果两个值不相等，则跳转到错误的标签 终止程序
0x00007f8e7cf9acfa 64 48 8b 04 25 28 00 00 00      movq    %fs:0x28, %rax
0x00007f8e7cf9ad03 48 8b 8c 24 e0 00 00 00      movq    0xe0(%rsp), %rcx
0x00007f8e7cf9ad0b 48 39 c8                      cmpq    %rcx, %rax
0x00007f8e7cf9ad0e 75 0a                      jne     0x7f8e7cf9ad1a          ; <+1098> at gpu_program_gles.cpp
.....
0x00007f8e7cf9ad1a e8 f1 6f 05 00              callq   0x7f8e7cff1d10          ; symbol stub for: __stack_chk_fail

// 你问我怎么知道0xe0存储的是保护值？我是从0x00007f953ff9ad03逆推出来的
// 在实验中肉眼可见，是否触发堆栈保护，rac 和 rax 的值确实会不一样
```

了解原理之后，我们可以轻易的知道，-fstack-protector会在函数退出时通知我们堆栈被破坏（修改）了

这意味着这种方式有两个缺点，无法发现越界读，无法实时（在当前指令处）立刻暴露内存错误

请再考虑一个问题：假如有一个病毒程序修改了0xf0(%rsp)，非0xe0(%rsp)，甚至修改了堆内存，-fstack-protector还有用么？

所以需要更强大的功能来保护内存：-fsanitize=address，地址消毒技术

网络上关于地址消毒的文章大多取自论文AddressSanitizer: A Fast Address Sanity Checker

所以我找了个AI翻译了这篇论文，花费了大约几个小时，边读边写下以下内容：

影子内存 shadow memory
 由于大部分系统都是8字节对齐的，即malloc返回的堆内存总是8字节对齐，所以可以使用最少1个字节来表示这8个字节的状态
 一般情况下这1个影子字节，如果是正值，表示对应的内存可寻址，如果是负值就是不可寻址（非法访问）
 在8字节对齐的内存系统中，影子地址的计算方式是 shadow_addr = addr >> 3 + offset，offset是一个不太可能被占用的进程空间地址

红区 redzones
 hook 内存分配函数 malloc free，在原本申请的基础上多申请一些内存作为红区（堆红区最小32字节）
 这样内存的排列就会变成：red mem red mem read
 比如原本申请8个字节，现在变成32+8+32-10+32个字节，但这些额外字节程序员不感知
 当这8个字节被上越界时，左边的红区会触发，而下越界时，则右边的红区会触发，例如：
 char foo[7];
 memset(foo, 0, 8);
 这种内存错误，并不一定会被cpu或内核发现（如果无法理解 请去查阅内存原理和memset等低级函数的源码）
 但在红区面前，简直小儿科，分分钟抓到
 但是！！！！以下这种代码，地址消毒就抓不到了：
 int *a = new int[2]
 int *u = (int*)((char*)a+6)
 *u = 1
 XD，笑死，本质上这个红区技术，无法处理刚好落在有效内存区的地址
 谷歌的工程师想过解决方案，但是由于太废性能直接放弃了

(论文还顺便黑了 Valgrind 和 Dr.Memory, 说他们也做不到)
(“这个技术苹果做不到 鸿蒙内部的压力也就没这么大了 XD”)

读完文章之后我其实还是对asan能处理越界读(out-of-bounds read)有一些疑惑
实际上还有几个例外场景, 例如结构体, 高性能检测是有代价的
不过我不想再进行下去, 大致了解一下asan的原理就差不多了

使用谷歌搜索"asan out-of-bounds read", 得到一些其他的编译选项, 如-fsanitize=bounds
可能asan发展了这么久, 早已新增了很多优化

还有一个额外收获是, 了解到了一些做AI或者计算的人可能更需要“超越界检测”这种技术
`x = y[MAX_LENGTH + n]` 在神经网络场景中, n超大越界

通过分析asan的原理, 有一个老问题得到了解释: 为什么有些进程启动时, 虚拟内存会占到几百G

这个问题我问过高章和岑友一, 他们说不太关心虚拟内存, 当时我认为这是进程异常的现象

我是错的, 一些技术会产生高额的虚拟内存

4)三方库工具, 由于时间关系, 就不再去研究, 尽管谷歌一直黑Valgrind慢

但是我认为, 理论上来说, Valgrind的逐指令检查的技术原理, 能最完善的发现内存错误

简单总结:

1)几乎没有性能损失, 无法自动化, 对使用人员有业务和技术要求

2)-fstack-protector我认为有几个百分点的性能损失、但只能单一的保护栈写入

4)极慢, 数倍性能损失

6)适合鸿蒙环境, 但可能业务开发拿到6)的堆栈之后一下子看不出问题, 还会用2)去试一遍

所以2)是比较受欢迎的一种方式, 平时开发时使用-fstack-protector简单保护

出问题时使用asan, -fsanitize=address去调试

实际案例: 使用1)找到华为自研编译器的优化bug

23年, 在一个华为自研硬件上适配矩阵计算相关的数学库, 程序输出的结果和intel机器对比总有一些差异

使用gdb调试器顺藤摸瓜, 在O2优化下找到了一个变量的内存, 在两行代码之间突变, 而O0优化则不存在

看了一下汇编(隐约记得)是一条加成融合指令有问题, 最终搜索O2背后的优化细节, 通过逐条增减编译指令

进行实验, 最终确认是一条对汇编文件排序优化的指令影响的(至今未明白原理)

指令和avx

英特尔汇编语法 64位的X86处理器有16个通用寄存器RAX, RBX, RCX, RDX, RDI, RSI, RBP, RSP and R8-R15 x86的通用寄存器有eax、ebx、ecx、edx、edi、esi。这些寄存器在大多数指令中是可以任意选用的 esp 保存每层函数调用的堆栈栈顶 ebp 栈底寄存器

QWORD PTR只是一个大小说明符（這裡是冗餘的，但它提高了可讀性），它與具有基址寄存器無關。

INC 加 1. test 与运算 猜测:if编译的一般结果 LEA 装入有效地址.例: LEA DX,string ;把偏移地址存到DX. cmp 比较

movsxd movups u 不必对齐, ps 单精度浮点数 mov rax,qword ptr [r13] 从r13拿到值, qword的意思是2x4个字节(x86) ptr的意思是值是地址 而不是值, 注意ptr的位置在第二个操作数前, 作用的对象是第二个操作数

cdq ; EDX = signbit(EAX) 这个指令不需要填操作数 有默认的结果保存在edx

IDIV 有符号除法 DIV 无 (EAX)<-(EDX,EAX)/(SRC) 的商 (EDX)<-(EDX,EAX)/(SRC) 的余数

edx = rax + rdx * 1 lea 0x3(%rax,%rdx,1),%edx ##### arm 汇编 cbz 寄存器 地址 寄存器存0就跳 stp : Store Pair , 用于将两个通用寄存器中的值存储到内存中 多用在函数开头入栈的时候 从sp指针连续取地址 8位递增 ADR 是一条小范围的地址读取伪指令,它将基于PC的相对偏移的地址值读到目标寄存器中。格式: ADR register,exper。ldr 取数据 ldaxr ldaxrb ldaxrh 从Memory地址addr中读取一个双字/字节/半字数据到目标寄存器Xt/Wt中,标记物理地址为独占访问 ldur 从Memory地址addr中读取双字/字节/半字/字数据到目标寄存器Xt/Wt中 带"S"表示需要符号扩展.立即数偏移 #simm9 = { -256 ~ +256 } 的任意整数,不需要对齐规则.

arm的 sub 减法 ##### 顺便把计算机的笔记也放到这

Compulsory 强制缺失 Capacity 容量缺失 看起来vector借用了这个概念 Conflict 冲突缺失 有点像片复制

读:直接找字副本 找不到就去下一级缓存 写:直写:直接将L1更新到L2或者更低级的缓存 写回:憋着,直到L1那一行即将被清理,需要维护dirty bit 写分配 (Write Allocate) 写分配法是把这个未在cache中的块加载到cache中后在进行写

非写分配 (No Write Allocate)
非写分配法指的是直接把这个块在主存中写不进行调块

缓存行的大小是64字节,难以想象 竟然这么小 以下网站提供了cache line的测试:测试结果大致是cache line 能够加载不同内存区域的数值 提供给cpu计算 <http://igoro.com/archive/gallery-of-processor-cache-effects/>

顺便把矩阵的知识放在这 arma::mat 在内存中的存储顺序: 优先列存储。mem[in_row + in_col*n_rows] mat mm(ptr, 6, 6); 在内存中: mm(1, 1) 和 mm(0, 1) 相邻 也就是说写法和我们操作二维数组是相反的 ##### gdb调试记录:

/data/user/x30015668/gdbtest/gdb/usr/local/bin/ maintenance info sections set args 启动参数 -nc 8 -f 222.hw -p 222 -d D:\tmp\hw_test bt f 跳帧 locals info thread info disassemble 汇编 help set scheduler-locking 单线程锁定调度 attach pid set environment LD_LIBRARY_PATH= gdb --batch --quiet -ex "thread apply all bt full" -ex "quit" disas /m fun set pagination off // 关闭询问 b *main+98 set print pretty on 结构体显示 p *结构体指针 或者 变量名字 show debug-file-directory 启动时 -s 指定文件

break pthread_create 查看进程启动时自动启动的线程位置 p 变量名 打印 set print pretty list l 查看源码 (后面可以接数字,表示查看该行附近的源码) next n (执行完该行后)跳到下一行,遇到函数直接执行完成 step s (执行完该行后)跳到下一行,遇到函数会进入 finish 运行到函数结束 continue c 继续运行 (如果后面没有断点,则一直执行到函数结束) break b 打断点 (d 行号,或d函数名) thread 28 仅28号线程触发断点 info breakpoints 显示断点信息 (比如目前有几个断点,是否被启用等等) delete d 删除断点 (delete 断点编号:删除某一断点,delete:删除所有断点) print p 打印变量值 (p 代码中变量名:打印代码中变量的值) info locals 打印当前所在函数局部变量的值 run r 启动程序 (设置完断点后用该命令启动程序) until u 执行到指定行 info i 显示信息 help h 显示帮助信息 (如:help list) quit q 退出GDB !(shell命令) 在GDB里执行shell命令,第4.2节会用到 shell (shell命令) 同上

实战 source /home/guest/SimMechSolver/stl-views-1.0.3.gdb set args -nc 16 -f case8_new.hw -p case8 -d /tmp/x30015668/hw -u asdd (gdb) b Mortar3DStandard.cpp:18 pvector (('TianJi::ContactFace')*sface)->face->nodes p ((TianJi::Node *)0xcabdd88)->id b Arrow2D.cpp:52 b Polygon2D.cpp:116 if i==1 && j==2

记录一次汇编调试

__libc_message 大概是个函数 无源码,获取其中一个变量的内存值 https://elixir.bootlin.com/glibc/glibc-2.17/source/sysdeps/posix/libc_fatal.c 63行获取环境变量,存放在on_2 on_2的地址在 rax寄存器

x/16xb 0x7fffffff69f 从这个地址开始 打印16个16进制数 x/16ub 0x7fffffff69f 从这个地址开始 打印16个10进制数

得到的10进制结果是: 0x7fffffff69f: 97 115 100 46 108 111 103 0 0x7fffffff6a7: 86 84 95 76 73 66 95 68 第一行的内容ascii合环境变量的结果 asd.log\0

63 const char *on_2 = __libc_secure_getenv ("LIBC_FATAL_STDERR"); 64 if (on_2 == NULL || *on_2 == '\0') 65 fd = open_not_cancel_2 (_PATH_TTY, O_RDWR | O_NOCTTY | O_NDELAY);

pragma GCC optimize("O3") gcc

```
0x00007ffff7192c5a <+202>: test %rax,%rax 0x00007ffff7192c5d <+205>: je 0x7ffff7192c68 <__libc_message+216> =>
0x00007ffff7192c5f <+207>: cmpb $0x0,(%rax) 0x00007ffff7192c62 <+210>: jne 0x7ffff7192e6d <__libc_message+733>
```

nm 获取二进制文件包含的符号信息 strings 获取二进制文件包含的字符串常量 strip 去除二进制文件包含的字符 readelf 显示目标文件详细信息 objdump 尽可能反汇编出源代码 addr2line -f -e 根据地址查找代码行 /proc/*pid*/coredump_filter 0x33 readelf -p .comment
objdump -s --section .comment SimMechSolver nm -A 观察调试信息
#####

pragma GCC optimize("O3") gcc

Position-independent executables (PIE) position-independent code PIC
<https://gist.github.com/reveng007/b9ef8c7c7ed7a46b10a325f4dee42ac4>

计算机体系结构 1、缓存性能提高 编译器优化 2.2.9 (1) 循环交换 for (int j = 0) -> for (int i = 0) x[i][j]的情况 for (int i = 0) for(int j = 0)

(2) 分块 在缓存容量不足的情况下，调整循环步长，优先集中处理子矩阵的元素

什么是向量：每个向量寄存器都是一个固定长度的寄存器组，每个向量寄存器保留64个元素 每个元素64位 向量操作高度重叠，将同样的数据发送到不同的向量寄存器

不定长的步长：截断整除 C写出来很丑。数据量不够的情况，效果低。

列加到行上：步幅不同，从寄存器设计的角度能解决，代码角度呢？

稀疏矩阵的向量化处理：集中--取得索引向量 分散--存储索引向量 集中：是指取得索引向量，并在此向量中提取元素，元素位置的确定是将基础地址加上索引向量中给定的偏移量。 分散：存储索引向量 由程序员发出指令告诉编译器 执行此动作（针对稀疏矩阵的复合操作？）

mumps 内存功能 job = -1 初始化 Initializes job = -2 中止 erminates job = -3 save / restore job = 1 执行分析 performs the analysis
job = 2 执行分解 performs the factorization job = 3 求解 computes the solution 目前代码中的顺序是 -1 1 2 3 -2

CAE方法进行工程分析： 1、客户输入需求 2、简化分析模型 前处理：网格划分 边界条件、约束施加 载荷 3、建立数学方程式 4、解方程求解器、FEM 复杂问题简化 子区域方程联立，先局部求解 再全局积分 5、绘制图表 6、试验验证

#####FEM：静态分析：应力 应变 位移 力在物体内部产生，构成物体所有力平衡

单元的类型也太多了 节点自由度，不同的单元 自由度类型不同

网络的每个节点的刚度矩阵是求解位移的关键，刚度矩阵是一个方阵 行列数=自由度数 2个单元的公共位移相同

#####显卡加速：英伟达3层接口 GPU程序流程：1、CPU分配空间 2、CPU复制数据 3、CPU加载内核给 4、CPU拷贝结果 4层内存速度 local shared global SM KERNEL 线程块 线程 假设有一个核函数里面有3个循环，每个循环100次，循环是串行的，那么一个线程块完成一个循环。？这里她讲得很不好 SM是用来处理线程的，里面有很多处理器

<<<1,N>>> 用N个线程跑1次

多处理器以 32 个并行线程为一组创建、管理、调度和执行线程，称为warps 归约

GPU实战： 1、如何编译异构程序 2、改造代码插入 流程解析 数据结构 3、thrust库

内存非法访问调试 用这个启动程序 cuda-memcheck

#####深度学习 数学：

机器学习中的关键组件 可以用来学习的数据 (data) 如何转换数据的模型 (model) 一个目标函数 (objective function) 用来量化模型的有效性 调整模型参数以优化目标函数的算法 (algorithm)

数据集由一个个样本 (example, sample) 组成，每个样本由一组称为特征 (features, 或协变量 (covariates)) 的属性组成。预测的是一个特殊的属性，它被称为标签 (label, 或目标 (target))

我们需要定义模型的优劣程度的度量，这个度量在大多数情况是“可优化”的，这被称之为目标函数 (objective function)

模拟得很好，但不能真正推广到测试，过拟合

深度学习流程： 1、输入 随机搞个w 或者多个w 2、使用w计算分数，卷积神经网络同，前向传播 3、激活函数 4、

分布式三方库 Legion Programming System Ray

valgrind 内存调试

ARM压栈的顺序依次为当前函数指针PC、返回指针LR、栈指针SP、栈基址FP、传入参数个数及指针、本地变量和临时变量

perf性能分析 perf report -g --call-graph perf record -g -F 999

(lldb) register read -a General Purpose Registers:

lldb 调试命令 pro hand -p true -s false SIGSTOP image lookup --verbose --address \$pc memory read --format hex --size 8 \$sp

ARM压栈的顺序依次为当前函数指针PC、返回指针LR、栈指针SP、栈基址FP、传入参数个数及指针、本地变量和临时变量

栈回溯流程 1、FP指向顶 而上一帧的FP通常在变量之前，也就是说先弹出所有的变量 才能找回FP 2、PC是指令记录 3、SP自动增长+4 4、lr是给ret用的

栈是自高向低的，这个逻辑非常淡疼，实际上使用的时候又是反向写，比如看这里的stp指令 最简单的场景：进来之前sp = 0x00000007f11adbb30 fp = 0x00000007f11adbb40 0x560c1f69bc <+0>: sub sp, sp, #0x20 这里把sp变成了10 0x560c1f69c0 <+4>: stp x29, x30, [sp, #0x10] sp + #0x10 = 20，执行完这一条之后0x7f11adbb20 - 0x7f11adbb28 存着fp 0x7f11adbb28 - 0x7f11adbb30 存着 lr 0x560c1f69c4 <+8>: add x29, sp, #0x10 这里把fp变成了20 当前栈帧是20 - 10 显然上一个FP保存在了当前栈帧之外？fp 是当前栈帧的底为啥是20啊 搞特殊是吧 0x560c1f69c8 <+12>: stur w0, [x29, #-0x4] -> 0x560c1f69cc <+16>: ldur w8, [x29, #-0x4]

memory read --format hex --size 8 \$sp 0x7f11adbb10: 0x00000007f11adbb40 0x000000070c1f6a1c 0x7f11adbb20: 0x00000007f11adbb40 0x000000560c1f6a24 0x7f11adbb30: 0x000000560c1f69f8 0x0000000000000007 0x7f11adbb40: 0x00000007f11adbb50 0x0000005b254099c4

lldb 启动命令

settings append target.exec-search-paths D:\rk3568\lib.unstripped\arkui\ace_engine

settings append target.symbol-search-path D:\rk3568\lib.unstripped\graphic\graphic_3d

settings set -- target.source-map /srv/workspace/ D:\for_debug

性能加速实践

并发研究

```
perf stat -r 1 -e instructions,L1-dcache-loads,L1-dcache-load-misses,itlb_misses,walk_completed,cache-misses,cache-references
taskset -c 24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47 ./foo -nc 24 -f 222.hw -p 222kp -d
.././222 -u 2zas
```

*****矩阵乘法 一般的 GEMM 可以分解为多个对 GEPP、GEMP 或 GEPM 的调用。这些本身可以分解为多个对 GEBP、GEPB 或 GEPDOT 核的调用。

GEBP 是 General Matrix-Matrix Multiplication (GEMM) 的基本操作之一 考虑将 $\text{matC}(m \times n) = \text{matA}(m \times k) * \text{matA}(k \times n)$ 其中matB matC分解为N个列块 假设A：缓存足够放得下B、C的列块和整个matA 假设B：如果B块 C块 A都在缓存中，则以CPU峰值速度计算 假设C：A持久在缓存

TLB对cpu断续计算的影响 比 cachemiss更大

非连续 复制到连续

为了将数据移动到寄存器中的效率最高，重要的是组织计算 第六章讲

第5章 全章讲 几种基础的算法实现 GEMP 怎么分配数据在L2 TLB 比较好

我们强调，在本小节的结论基于一个观察，即在几乎所有当前的处理器上，对于L2缓存来说，块操作是有优势的。如果例如，L1缓存的块操作更优选，那

Duplicate（重复）。此参数指示矩阵B的元素是否在B的打包过程中进行了复制。这是为了利用Pentium4（Northwood）和Opteron处理器上的SSE2

持续带宽（Sustained Bandwidth）。这是从指定的存储层到寄存器的观察到的持续带宽，以double/cycle为单位。就是缓存一个周期能读写多少个浮点数

覆盖区域（Covered Area）。这是TLB可以访问的内存大小。一些架构具有（速度较慢的）级别2 TLB，它的功能相对于L1 TLB与L2缓存的关系与L2缓

*****矩阵乘法 第一章 SISR 单指令 单数据 MISD 没人这么玩 SIMD GPU MIMD 任务级 task

lscpu | grep "CPU(s)"

Amdahl 定律：通过改进计算机某个部分而获得的性能增益 $\text{加速比} = \frac{\text{整个任务在采用该升级时的性能}}{\text{未采用该升级时的性能}}$

$\text{新执行时间} = \text{原执行时间} * ((1 - \text{升级比例}) + \text{升级比例} / \text{升级加速比})$ 总加速比 = 原执行时间 / 新执行时间

还有一些其他公式 用到就记下来

第五章 TLP 线程级并行

SMP -> UMA DSM -> NUMA

远程访问带来的长时间延迟：分离核心之间的数据通信 35 - 50 周期 分离CPU之间的数据通信 代价就更高了100 - 500

openblas比intel性能差的原因：

1、硬件亲和性（之前在鲲鹏、x86下进行测试比较 从来没有设置过进程（线程）绑定核心）（为什么？因为效果提升不明显 mkl、kml内部，或者编译器应该都有做相应的优化）

使用taskset指定cpu id进行单次计算任务：taskset -c 24,25,26,,,47 foo -nc 24 -f ,,, 此法避免多进（线）程程序绑定到不同的NUMA域（2个物理CPU），从而产生“分离核心之间的数据通信”

openblas case27 137 -> 117s mkl case27 76 -> 72s

2、软件亲和性 -> intel hpc生成的汇编更漂亮

从应用层角度来看，完成单个任务（综合算例）所需要的C++代码、内存量 是固定的。实际上整个流程中，cpu使用到的资源、做出的行为，在不同编译器和硬件结构下，是不一样的

x86 -> 两种不同编译器 生成的二进制 的对比

初步证据：openblas版本 执行 222 算例时，222 ca27

指令数(instructions:u)相比于intel(696,106,213,727)和鲲鹏(623,662,640,014)多3倍(2,053,613,051,013) 此数据会在不同核数下有所波动，鲲鹏波动较小

L1-dcache-loads:u 相比于intel(229,678,254,434)和鲲鹏(231,906,260,214)多3倍(606,070,208,156)

一些猜想： avx没开？ cache-references:u疑惑：intel(3,691,994,630)和鲲鹏(249,451,144,526) openblas (8,472,985,701) 可能的解释：intel更少的L3缓存引用次数、更高的L1引用次数，比openblas快 -> 优化，(1)直接把数据提取到L1、L2，(2)每次引用更多的数据，(3)有更适配硬件的调度算法，标记数据热度 指令数少，缓存使用当然少

进一步证据： 1、首先重新编译一下openblas 显式打开 avx （编译失败？？） 需要加入一些编译选项 启动avx：-mavx512f -mavx512dq -mavx512vl -mfma 结果更慢了，需要去openblas的论坛问问

观察测试矩阵乘法的demo，intel版本的指令数（27,417,548）比openblas更高(13,684,860)？？

2、gdb调试观察同样的gcc代码生成的汇编

```
make CC=/usr/local/bin/gcc FC=/usr/local/bin/gfortran BINARY=64 NO_STATIC=1 BIGNUMA=0 NO_AVX512=0 NO_AVX=0 NO_AVX2=0 COMMON_OPT="-O2 -mavx512f -mavx512dq -mavx512vl -mfma" DEBUG=0
```

实验一：编译openblas时 显式启动AVX， make NO_AVX=0 cc=gcc 加上之后编译报错，需要手动加上-mavx512等编译选项，才能顺利完成编译

感觉是破坏了openblas自动监测环境的逻辑，或者openblas对6248R这个cpu支持有问题？
在openblas的target list里面没搜到6248R的架构，Cascade Lake，
只能搜到SKYLAKEX，实际上最终的编译产物也是按照SKYLAKEX编译的（得到 libopenblas_skylakexp-r0.3.26.so）。

实验结果：demo 的指令数量变多了（11,576,525 -> 13,885,862）时间也变长（0.02 -> 0.09）了 也就是说强行加avx512作用不大
foo 的结果和demo类似。指令数量变多，综合时间慢了0.5-1s

得去社区问一下

实验二：使用默认版本的openblas 变量是2个不同的foo（intel 和 mpicc编译）

我加了一些代码，让程序跑到稀疏矩阵求解的地方直接停止进程

intel版本的指令 87,976,258,251 比 mpicc版本 285,060,925,544 少很多

而缓存引用和命中次数，是相近的

38,945,564	cache-misses:u	#	23.829 % of all cache refs	(83.34%)
163,437,607	cache-references:u			(66.68%)

使用一个超级简单的for循环，把3级缓存的 基准线程并行效率 搞出来！！！！

未命名的页面(1)

.keep

